

問：

Format 2 指令的暫存氣可以寫 SW PC 嗎

在 SIC/XE 的標準語法中，**答案是不行的**。

雖然硬體層面上 PC 和 SW 確實是暫存器，但在 Format 2 指令（如 ADDR, COMPR, TIXR 等）的設計與組譯慣例中，這兩個暫存器通常不被允許作為運算對象。

以下是具體原因與技術細節：

1. 暫存器編號的限制

Format 2 指令佔用 **2 Bytes**，其中第一個 Byte 是 Opcode，第二個 Byte 拆解為兩個 4-bit 的欄位，用來存放暫存器的**編號（Number）**。

\$\$Format\ 2: [Opcode (8)] [r1 (4)] [r2 (4)]\$\$

SIC/XE 的暫存器編號如下：

編號	暫存器名稱	用途	是否常用於 Format 2 ？
0	A	累加器 (Accumulator)	是
1	X	索引暫存器 (Index)	是
2	L	連結暫存器 (Linkage)	是
3	B	基底暫存器 (Base)	是
4	S	一般用途 (General)	是
5	T	一般用途 (General)	是
6	F	浮點累加器 (Floating Point)	僅限浮點指令
8	PC	程式計數器 (Program Counter)	否
9	SW	狀態字組 (Status Word)	否

2. 為什麼不能寫 PC 或 SW ？

PC (Program Counter) 的特殊性

- **硬體自動控制：** PC 的值是由 CPU 根據執行進度自動更新的。
- **風險過高：** 如果允許使用 ADDR S, PC 這樣的指令，等於是在執行過程中隨意跳轉程式碼，這會破壞原本的控制流（Control Flow）。在 SIC/XE 中，改變執行順序應該使用 J, JEQ, JLT 等跳轉指令，而不是透過算術指令修改 PC。

SW (Status Word) 的特殊性

- **唯讀與條件標籤：** SW 存放的是比較結果（CC, Condition Code）和系統控制位元。
- **專用性：** 修改 SW 必須透過 COMP (比較指令) 或特定的系統調用，如果用 CLEAR SW 或 ADDR A, SW 來隨意修改，會導致條件跳轉（Jump）邏輯完全失控。

3. 組譯器的行為

如果你在原始碼中寫：

程式碼片段

ADDR PC, A (錯誤範例)

大部分的 SIC/XE 組譯器（如 Beck 所著教科書中的標準版本）會直接拋出 "Invalid Register" 錯誤。

總結

在撰寫 SIC/XE 程式或考題時，Format 2 指令可以使用的暫存器建議僅限於：

A, X, L, B, S, T (編號 0-5)

PC (8) 與 **SW (9)** 雖然有編號，但它們是給 CPU 內部使用的「特殊功能暫存器」，不參與一般的暫存器對暫存器運算。